

BACKEND EXPLANATION DOCUMENT

- **System architecture & request lifecycle:**

The backend for this project “University Waste Management System” is made mainly by node.js and express.js. This system is divided into several different layers that makes it more organized, clean, secure and easier to tackle.

Layers of the system:

- **Route Layer:**

It mainly defines API end points like:

- POST/api/v1/auth/login
- GET/api /v1/workers
- POST/api v1/waste/collect

Routes doesn't have system logic , their main function is to send requests to the controller. it deals with URLs

- **Middleware Layer:**

It mainly runs before controller, it handles:

- **JWT Authentication**-checks whether user is logged in by verifying the token sent in request ,if invalid then request is rejected.
- **RBAC Authorization**-checks user's role to make sure they have permission to access APIs
- **Invalid Validation**-verify data's correctness and secure it.

- **Controller Layer:**

it mainly contains the main logic. Controllers:

- Receive requests from routes
- Run SQL queries
- Process data and send JSON responses

- **Database Layer:**

Uses mysql in which queries are written and handled for better transactions.

Request Flow example:

- Client sends POST /api/v1/waste/collect.
- JWT middleware checks login.
- RBAC middleware checks role permissions.
- Controller inserts the waste record and updates dustbin status.
- Response is sent as JSON.

This keeps system secure.

- **Authentication flow & RBAC enforcement**

This use JWT for authentication and RBAC for security.

Role	Description
manager	Watch whole system
admin	Handle workers and records
cleaner	Collect waste

Login Process:

- User sends CNIC and pass to POST/api/v1/auth/login
- Backend retrieves from db
- Password is checked with bcrypt.compare().
- If correct,JWT token is generated with ID and role
- Token is returned and then used as headers of all future request

RBAC enforcement:

- Middleware reads role from token and checks if user can access the endpoints
e.g GET/WORKERS – Manager,Admin
- If not allowed,HTTP 403 forbidden

• Transaction Flows & Rollback Conditions

The system uses **database transactions** to keep data consistent when multiple related operations are performed.

Example - Waste Collection Transaction

Steps:

- Insert a record into waste_collection.
- Update the dustbin fill level.

```
BEGIN TRANSACTION;
```

```
INSERT INTO waste_collection ...
```

```
UPDATE dustbin ...
```

```
COMMIT;
```

Rollback:

If any step fails :

- All changes are undone (ROLLBACK)
- Dustbin status remains the same
- Error is logged

This ensures the database always stays correct.

- ## Raw SQL & connection pooling strategy

Raw sql was used instead of orm

- Full control over queries
- Easier transaction
- Better performance
- Safe queries to prevent sql injection:

```
SELECT worker_id, name FROM worker WHERE CNIC = ?;
```

Connection Pooling:

- A pool of reusable connections is maintained (limit: 10)
Benefits:
 - Faster response
 - Less server load
 - Supports multiple users at the same time

- ## Key design decisions & tradeoffs

- **REST APIs** – REST was chosen because it is simple, and easy to integrate with both web and mobile apps.
- **JWT instead of Sessions** – JWT was used because it works well for APIs, making the system easier to manage.
- **Raw SQL instead of ORM** – Raw SQL was preferred to have better control over queries and database transactions, and to clearly understand how data operations are executed.
- **Middleware for Security** – Authentication and RBAC checks are handled through middleware so security rules remain consistent across all API endpoints.
- **Swagger Documentation** – Swagger provides interactive API documentation at /api-docs, which makes it easier to test endpoints and understand how the API works.

Conclusion:

The backend was designed to be secure and reliable .it manages important tasks like user authentication,login,transaction and much more.Raw SQL was used to control database queries and to handle multiple reuquests.API doc which makes testing and understanding way more easier.Overall this is made on a small scale but can be expanded in the future.